CarritoGT

Migración de un carrito de compra en PHP a una API REST con Spring Boot

Dato	Valor
Estudiante	Felipe Adolfo Gómez Fajardo
Carné	3490-25-11732
Curso	Programación 2
Catedrático	Ing. Moisés Velásquez
Fecha	17 de octubre de 2026
Versión del documento	1.0
Repositorio	https://github.com/IngFelipe77/proyecto_carrito_compras

2. Evidencia de virtualización

Utilicé VirtualBox como hipervisor sobre Windows 11 (el anfitrión). Creé la máquina virtual «Mv_Lubuntu_FelipeG» con sistema invitado Lubuntu 24.04.4 LTS de 64 bits. La máquina virtual y Windows funcionan al mismo tiempo: el escritorio de Lubuntu se ve dentro de una ventana de VirtualBox, mientras Windows sigue abierto con el navegador, el editor y el resto de mis programas.



Figura 1. VirtualBox: configuración de la máquina virtual Mv_Lubuntu_FelipeG.

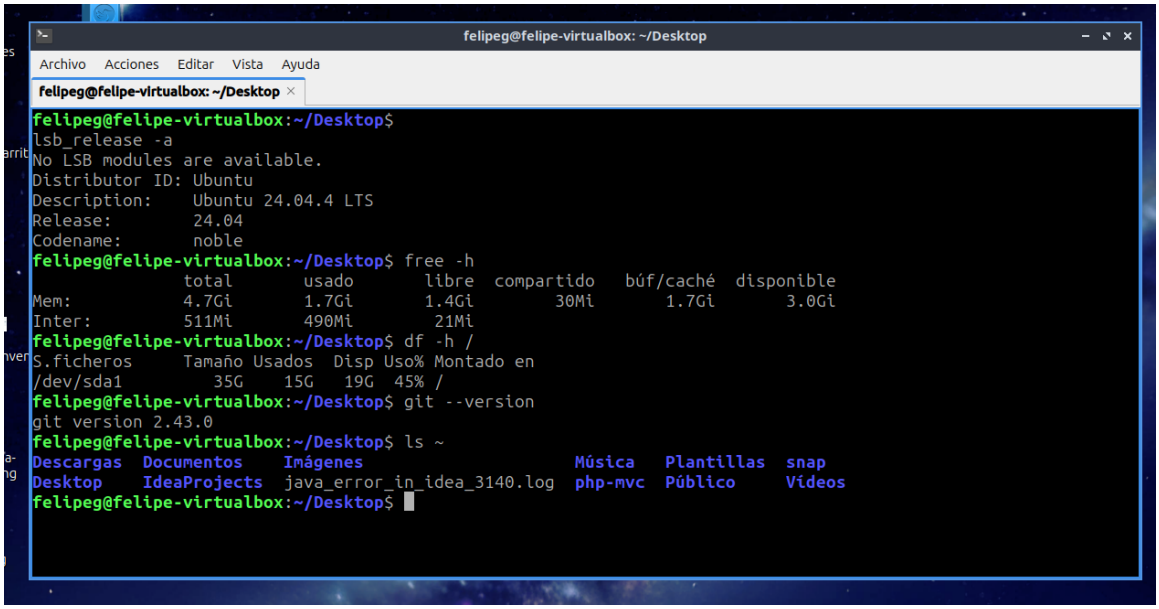


Figura 2. Dentro de la VM: Ubuntu 24.04.4 LTS, memoria, disco y Git 2.43.0.

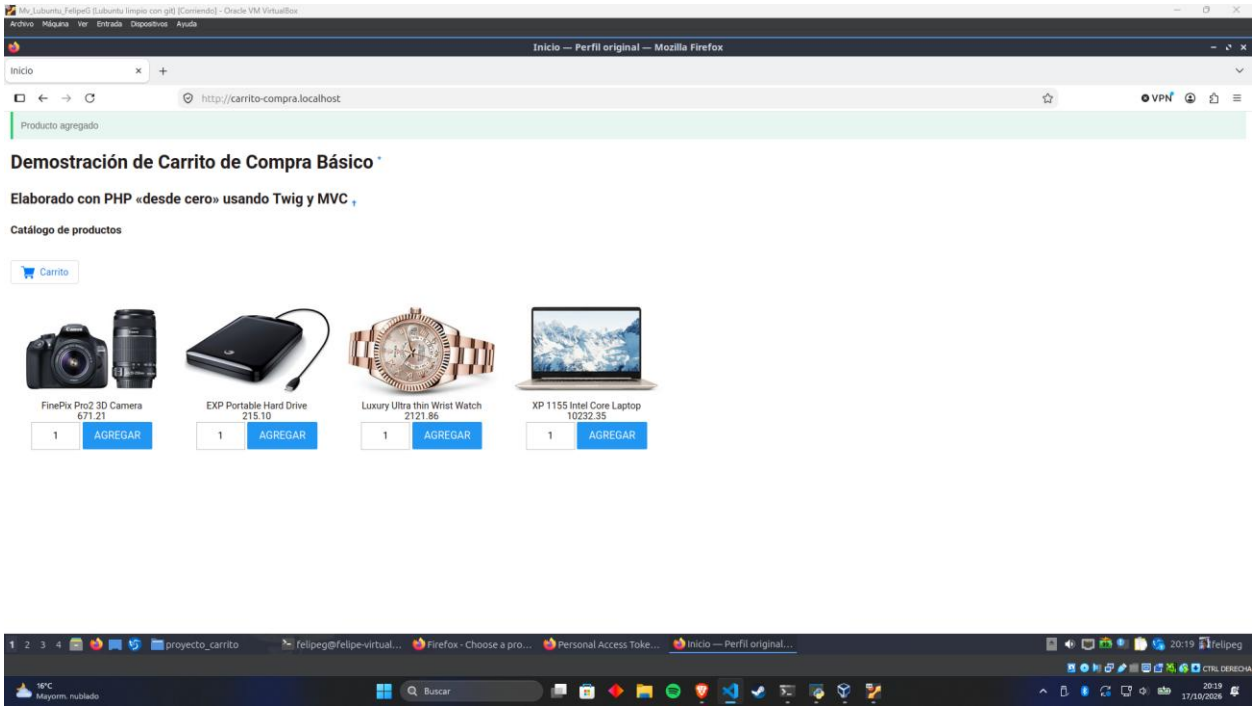


Figura 3. La máquina virtual en ejecución (ventana de VirtualBox) al mismo tiempo que Windows (barra de tareas de Windows abajo).

3. Justificación de recursos

El equipo anfitrión es una laptop con procesador Intel Core i5-12450H (8 núcleos, 12 hilos) y 7.7 GB de RAM. Dimensioné la máquina virtual para que corra Apache, MySQL, Spring Boot y el navegador sin quitarle a Windows lo necesario para seguir funcionando.

Recurso	Asignado	Justificación
Memoria RAM	5036 MB (≈ 5 GB; Linux ve 4.7 GiB)	Apache, MySQL, Maven con Spring Boot y Firefox corren a la vez dentro de la VM. Con 5 GB funciona bien y a Windows le quedan unos 2.7 GB. Cuando además abrí IntelliJ la VM se puso lenta, así que cerré programas de Windows y de la VM que no necesitaba.
CPU	4 procesadores virtuales	Maven compila y Spring arranca con varios hilos; 4 núcleos de los 8 físicos dejan el resto a Windows.
Almacenamiento	Disco virtual VDI de 35.70 GB (el sistema ve 35 GB, 15 GB usados)	Alcanza para Lubuntu, Java, Maven, MySQL, las librerías descargadas y el repositorio. El archivo VDI crece según el uso.
Red	Adaptador NAT (Intel PRO/1000)	La VM sale a Internet (descarga de paquetes, GitHub y Maven) a través de la conexión de Windows. Todos los servicios se prueban dentro de la misma VM, por eso no necesité reenviar puertos.
Video	128 MB	Suficiente para el escritorio y el navegador.

Ajuste de estabilidad: el sistema usa un poco de memoria de intercambio (swap de 511 MB). Para no saturar la VM, ejecuto Maven en la terminal y cierro IntelliJ cuando solo necesito probar la API.

4. Evidencia del servidor

Dentro de Lubuntu quedaron instalados y activos Git, Apache, PHP y MySQL. El enunciado acepta MySQL o MariaDB; usé MySQL 8.0.

Componente	Versión	Cómo se comprueba

Git	2.43.0	git --version
Apache	2.4.58	systemctl status apache2
PHP	7.4.33 (módulo php7_module de Apache)	php -v ; apache2ctl -M | grep php
MySQL	8.0.46	systemctl status mysql ; mysql -u root -p -e "SHOW DATABASES;"
Java / Maven	OpenJDK 21 / Maven 3.8.7	java -version ; mvn -v

El proyecto base requiere PHP 7 (RNF-04) y su manejador de errores convierte los avisos en excepciones, por lo que con PHP 8 la página fallaría. La máquina traía PHP 8.3, así que agregué el repositorio ondrej/php, instalé PHP 7.4, desactivé el módulo de PHP 8.3 y activé el de PHP 7.4.

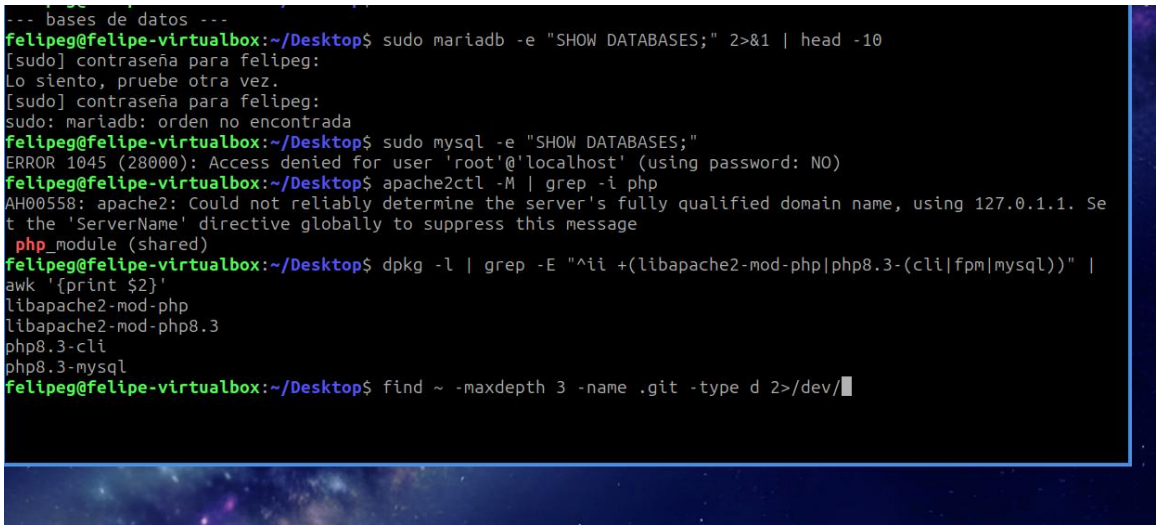


Figura 4. Estado inicial: Apache con PHP 8.3.

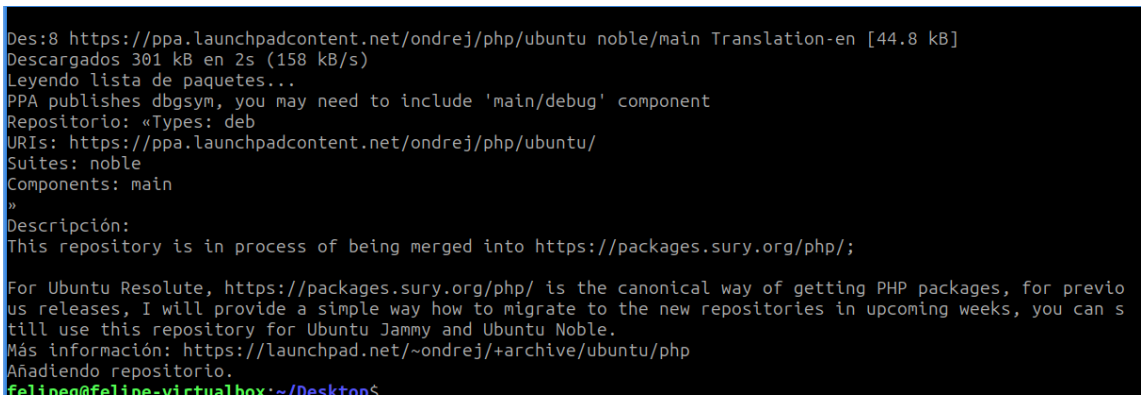


Figura 5. Se agrega el repositorio ondrej/php para poder instalar PHP 7.4.

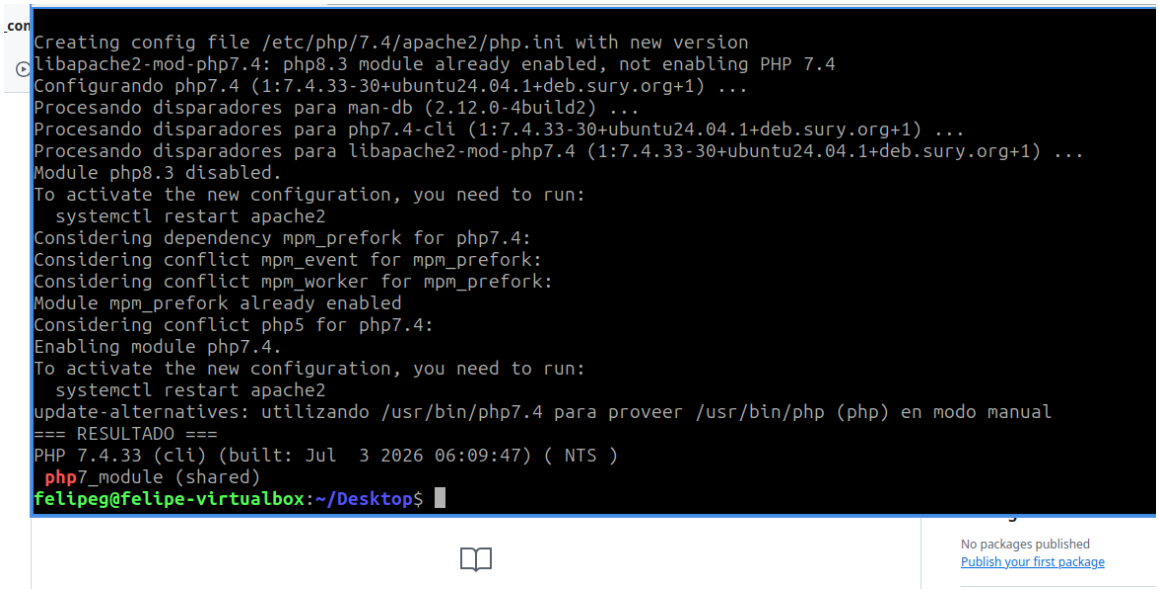


Figura 6. PHP 7.4.33 instalado y Apache con php7_module.

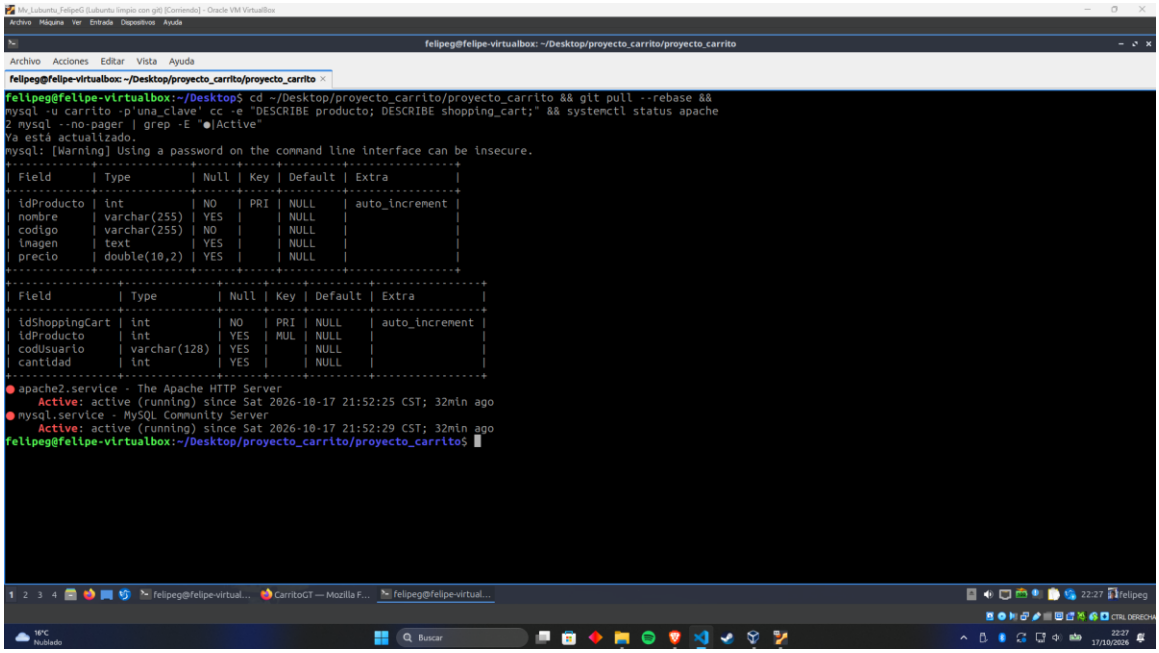


Figura 7. Apache y MySQL activos (active).

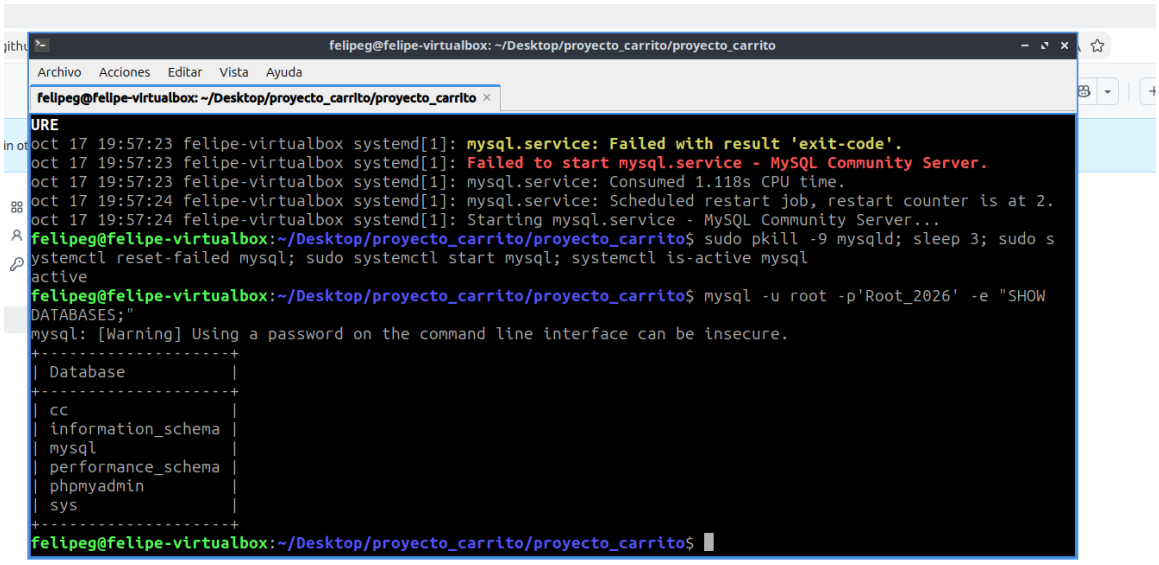


Figura 8. MySQL activo: la base cc existe junto a las bases del sistema.

5. Configuración del Virtual Host

Configuré dos Virtual Hosts en Apache. El primero sirve la versión PHP original; el segundo sirve la página nueva que consume la API.

Virtual Host	Dirección	DocumentRoot
carrito-compra.localhost	http://carrito-compra.localhost	/home/felipeg/Desktop/proyecto_carrito/proyecto_carrito/php/public
carritogt.localhost	http://carritogt.localhost	/home/felipeg/Desktop/proyecto_carrito/proyecto_carrito/frontend

Archivo /etc/apache2/sites-available/carrito-compra-localhost.conf:

```
<VirtualHost *:80>
    ServerName carrito-compra.localhost
    DocumentRoot /home/felipeg/Desktop/proyecto_carrito/proyecto_carrito/php/public

    <Directory /home/felipeg/Desktop/proyecto_carrito/proyecto_carrito/php/public>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/carrito-compra-localhost-error.log
    CustomLog ${APACHE_LOG_DIR}/carrito-compra-localhost-access.log combined
</VirtualHost>
```

Otros cambios que hice para que funcionara:

- AllowOverride All y el módulo rewrite (a2enmod rewrite): el .htaccess del proyecto manda todas las direcciones a index.php.
- /etc/hosts: se agregó la línea 127.0.0.1 carrito-compra.localhost carritogt.localhost.
- Permisos: chmod o+x sobre /home/felipeg y /home/felipeg/Desktop, para que Apache (usuario www-data) pueda llegar hasta mi carpeta, y escritura en php/logs.
- Se desactivó el Virtual Host viejo carrito-compra.conf, que apuntaba a una copia antigua del proyecto.
- php/App/Config.php: DB_USER = 'carrito', DB_PASSWORD = 'una_clave' (en lugar de root) y HOST = 'carrito-compra.localhost' (antes carrito-compra.com, lo que redirigía fuera de mi sitio).

Prueba de acceso: curl devuelve «HTTP/1.1 200 OK» y el navegador muestra la tienda PHP, con la que agregué y eliminé productos.

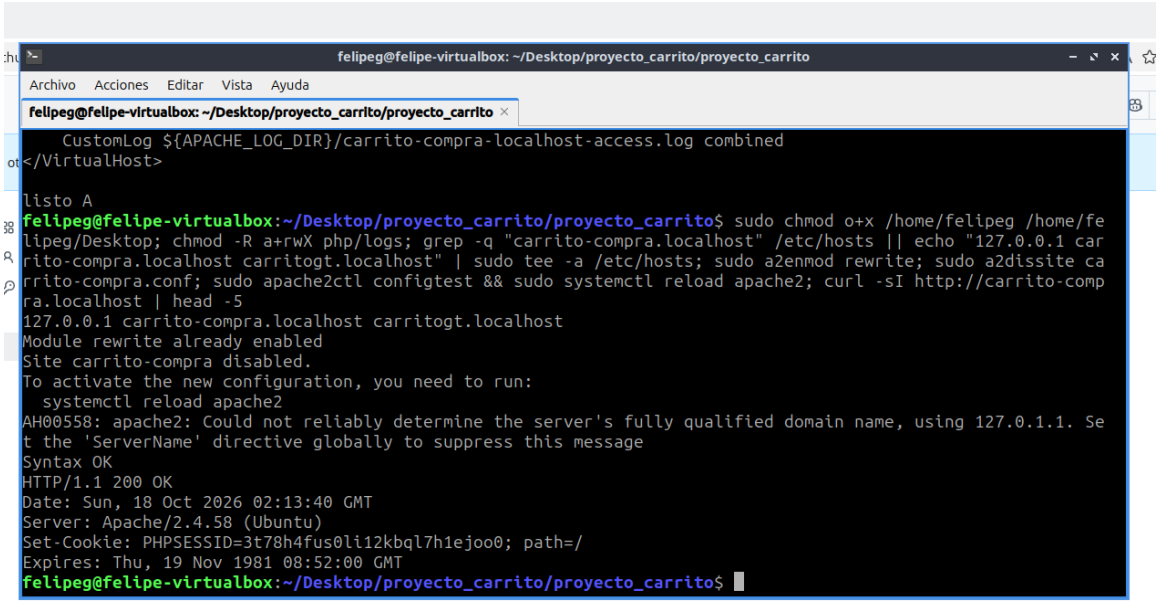


Figura 9. Prueba de acceso al Virtual Host: HTTP 200 y cookie de sesión PHP.

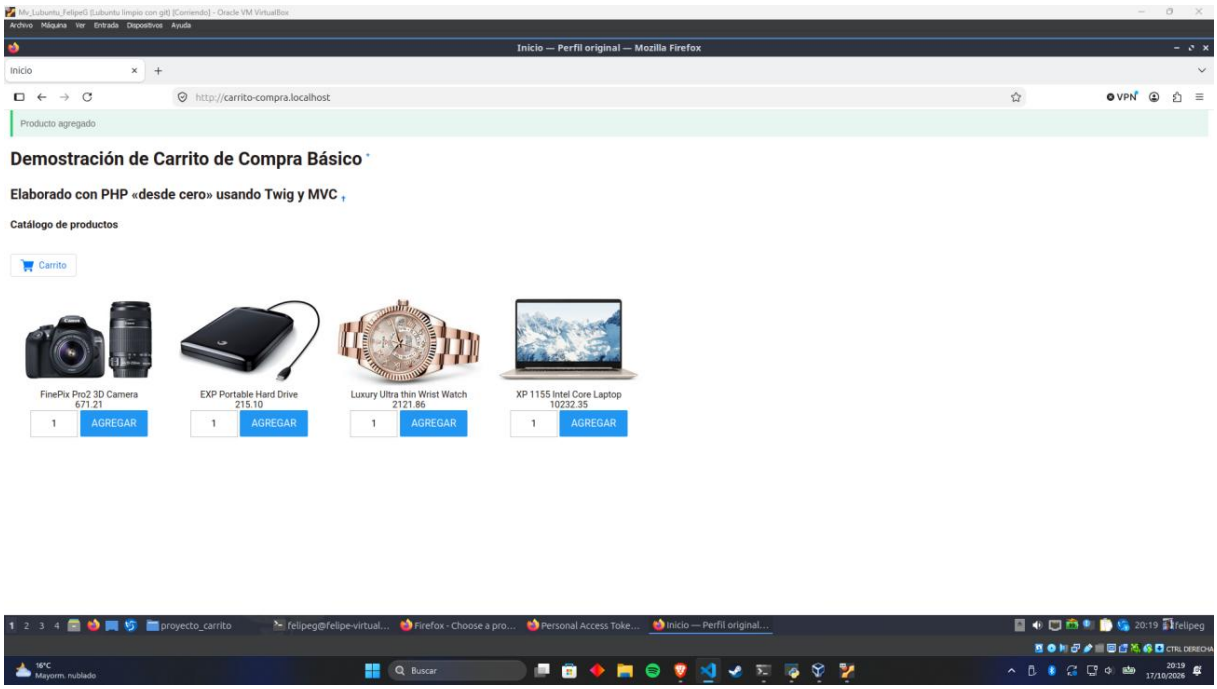


Figura 10. La tienda PHP en carrito-compra.localhost con el mensaje «Producto agregado».

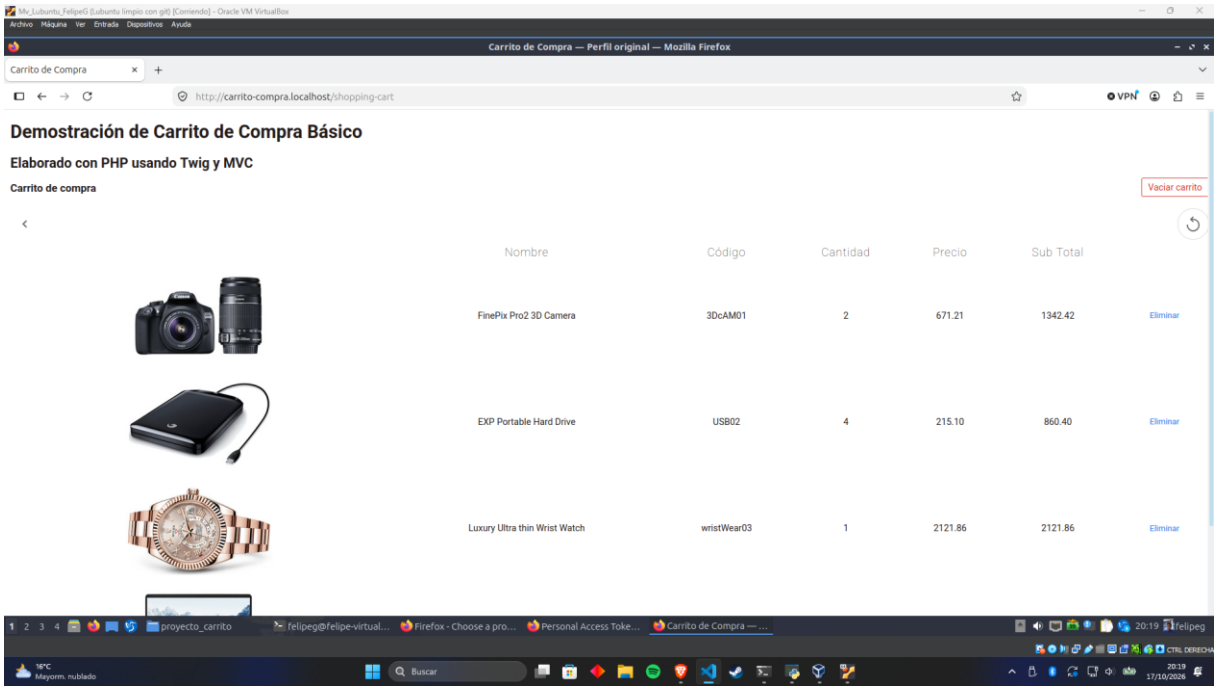


Figura 11. El carrito PHP con sus renglones y subtotales.

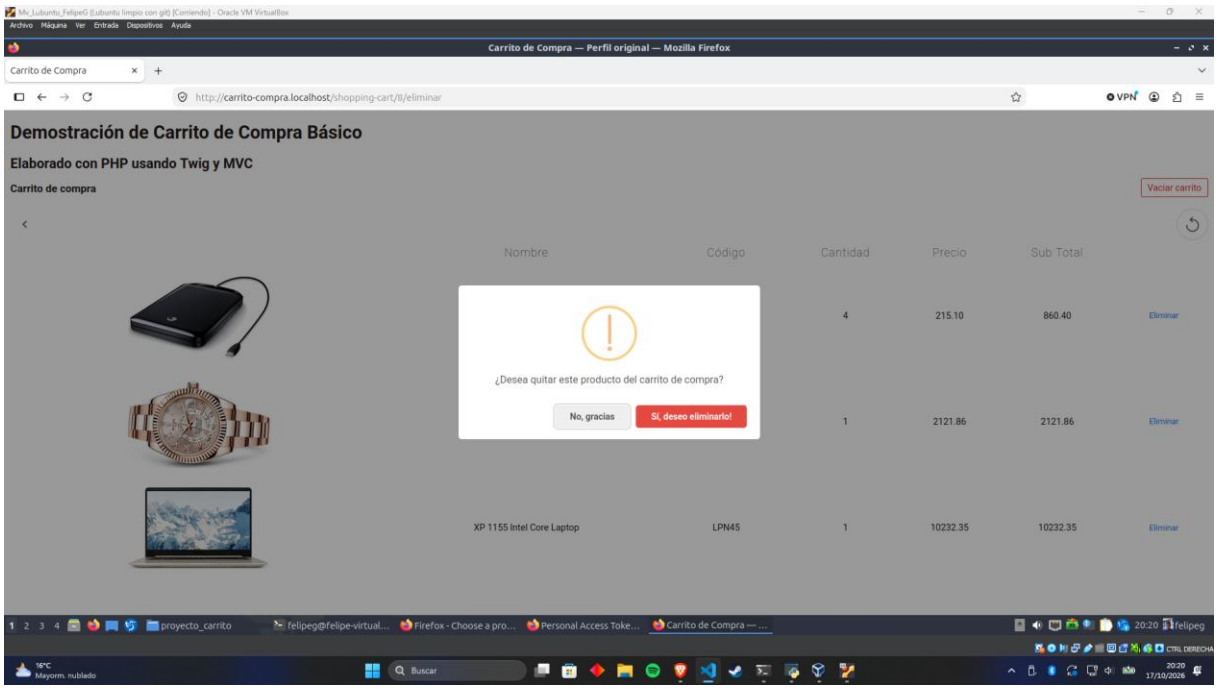


Figura 12. Confirmación al eliminar un producto.

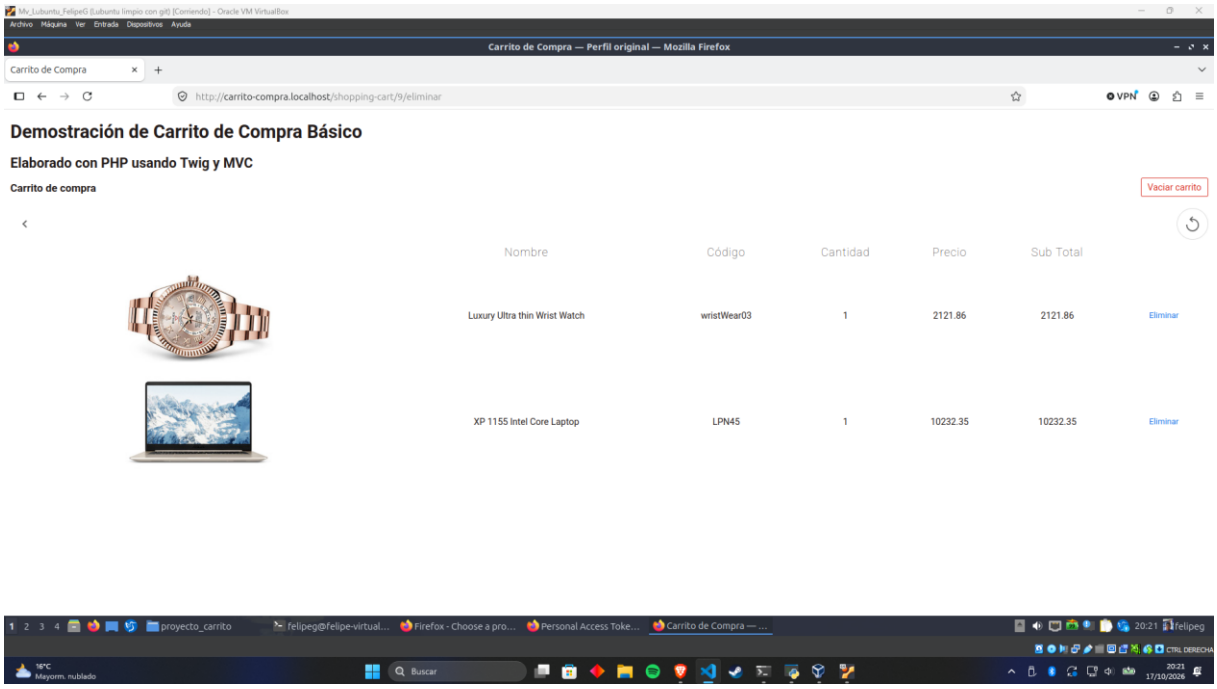


Figura 13. El carrito después de eliminar.

6. Diagrama de casos de uso

El actor principal es el Cliente. Funciones: ver productos, agregar un producto al carrito, ver el carrito, eliminar un producto del carrito y vaciar el carrito. Lo dibujé en Dia antes de empezar a programar.

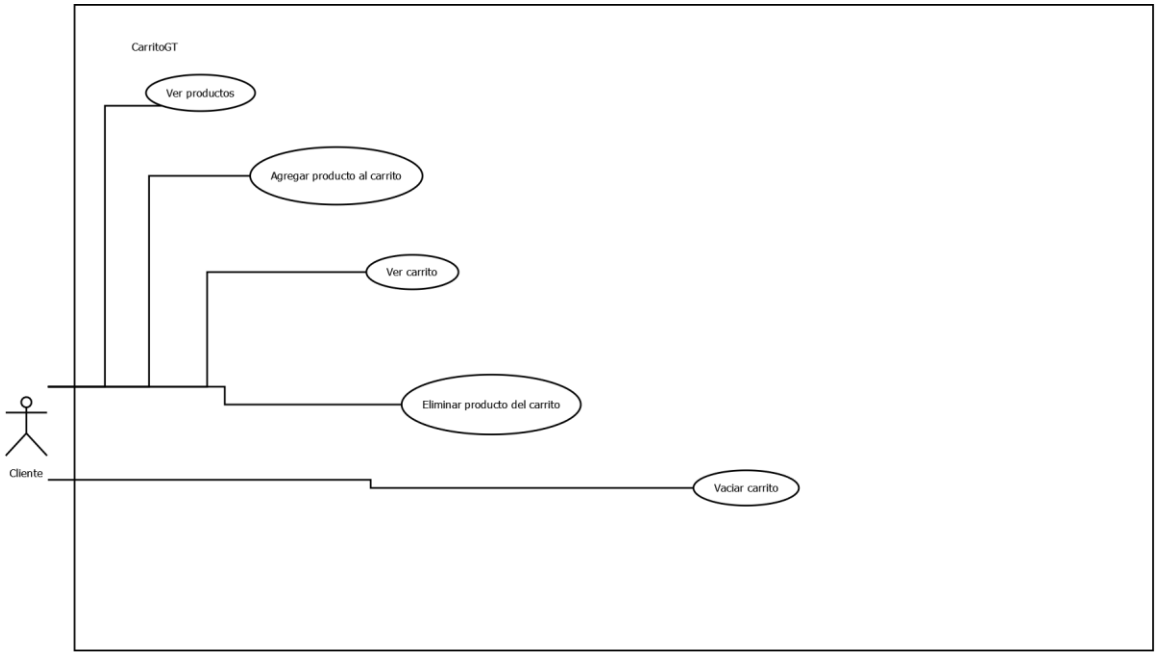


Figura 14. Diagrama de casos de uso de CarritoGT (Dia).

7. Diagramas de arquitectura

Versión PHP: el navegador pide una dirección, Apache la manda a index.php, el Router elige el controlador, el modelo consulta MySQL con PDO y Twig arma la página HTML completa en el servidor.

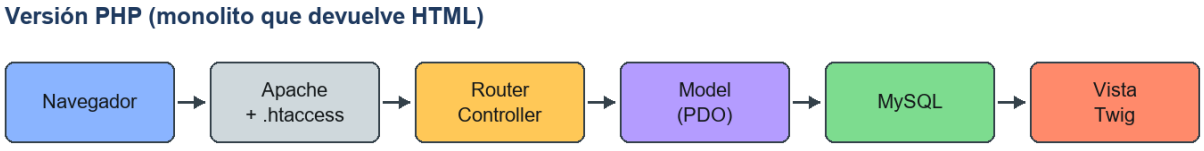


Figura 15. Arquitectura de la versión PHP.

Versión Spring Boot: la página es HTML y JavaScript estáticos. Con fetch pide datos a la API, que los devuelve en JSON, y JavaScript dibuja la página. La API está organizada en cuatro capas.

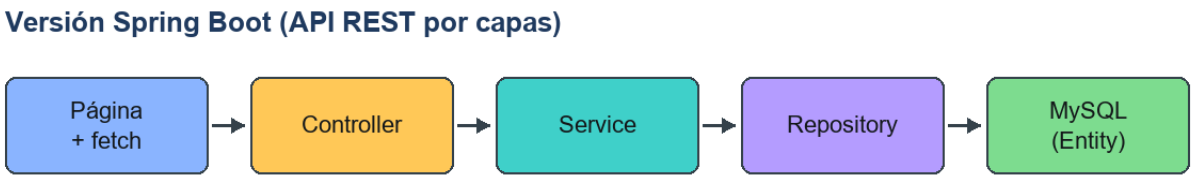


Figura 16. Arquitectura de la versión Spring Boot.

Capa	Paquete	Qué hace
Controller	controller	Recibe la petición HTTP y responde JSON con su código de estado. ProductoController, CarritoController y AgregarRequest (el cuerpo del POST).
Service	service	Aplica las reglas: valida la cantidad, comprueba que el producto exista y suma si ya estaba en el carrito. ProductoService y CarritoService.
Repository	repository	Interfaces que extienden JpaRepository; Spring Data genera la implementación y el SQL. ProductoRepository e ItemCarritoRepository.
Entity	entity	Clases que representan las tablas: Producto (producto) e ItemCarrito (shopping_cart).
Config	config	CorsConfig: permite que la página (http://carritogt.localhost) llame a la API.

Diagrama de clases (vista de implementación), dibujado en Dia antes de programar; el código sigue este diagrama:

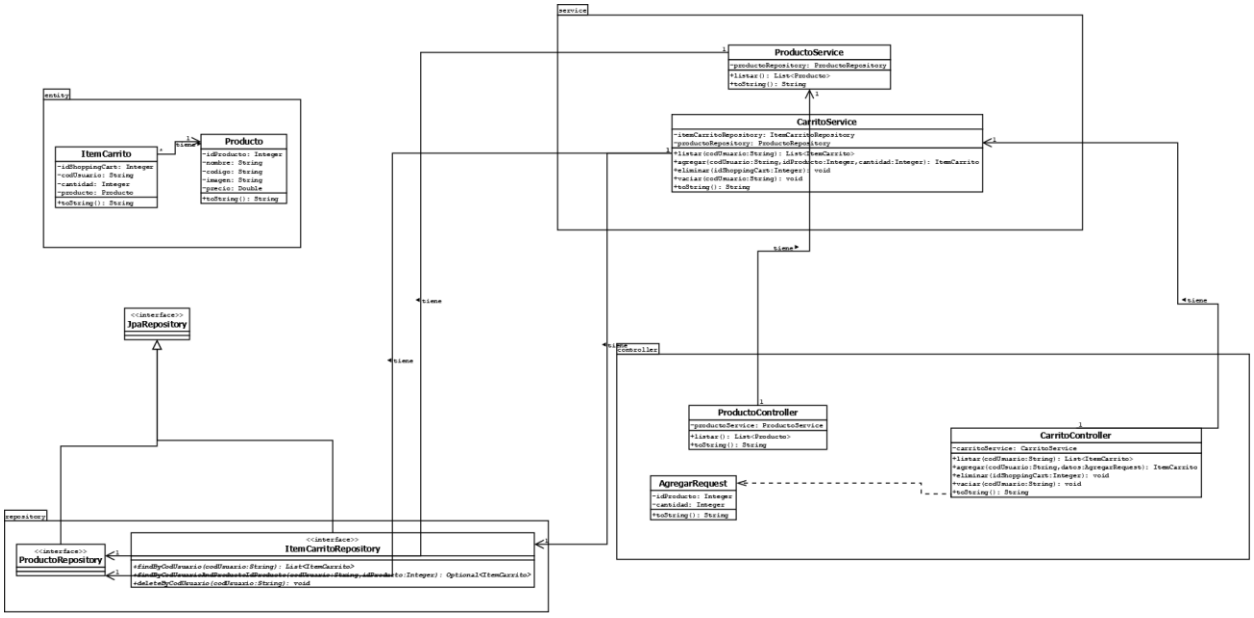


Figura 17. Diagrama de clases de la API, por capas.

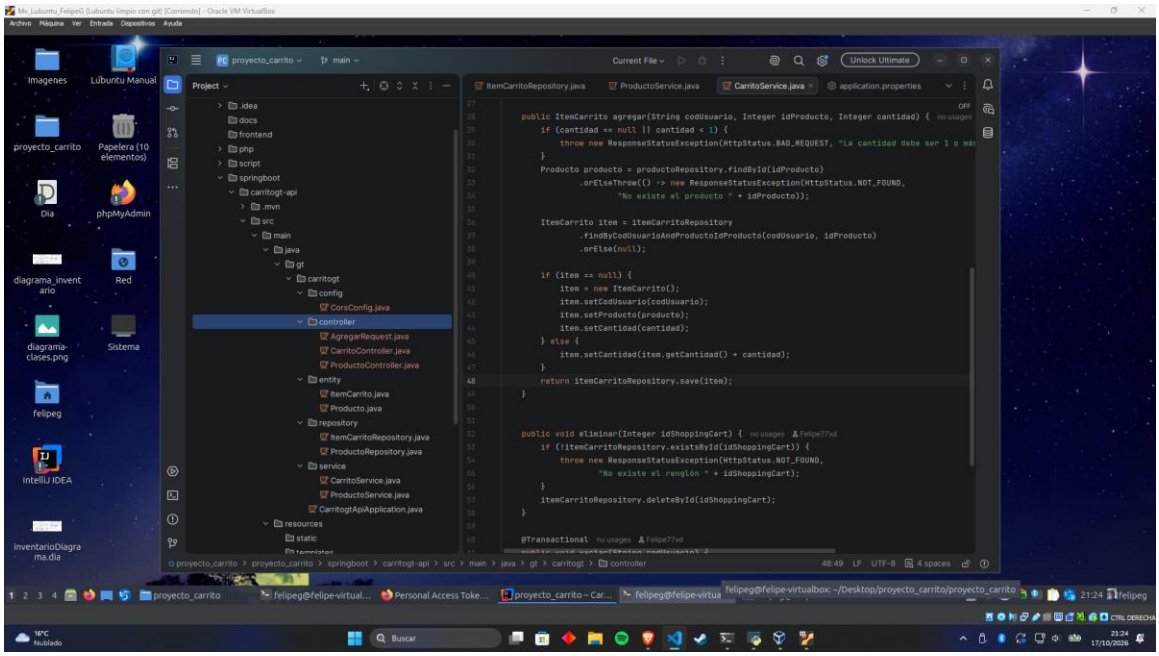


Figura 18. Los paquetes de la API en IntelliJ: config, controller, entity, repository y service.

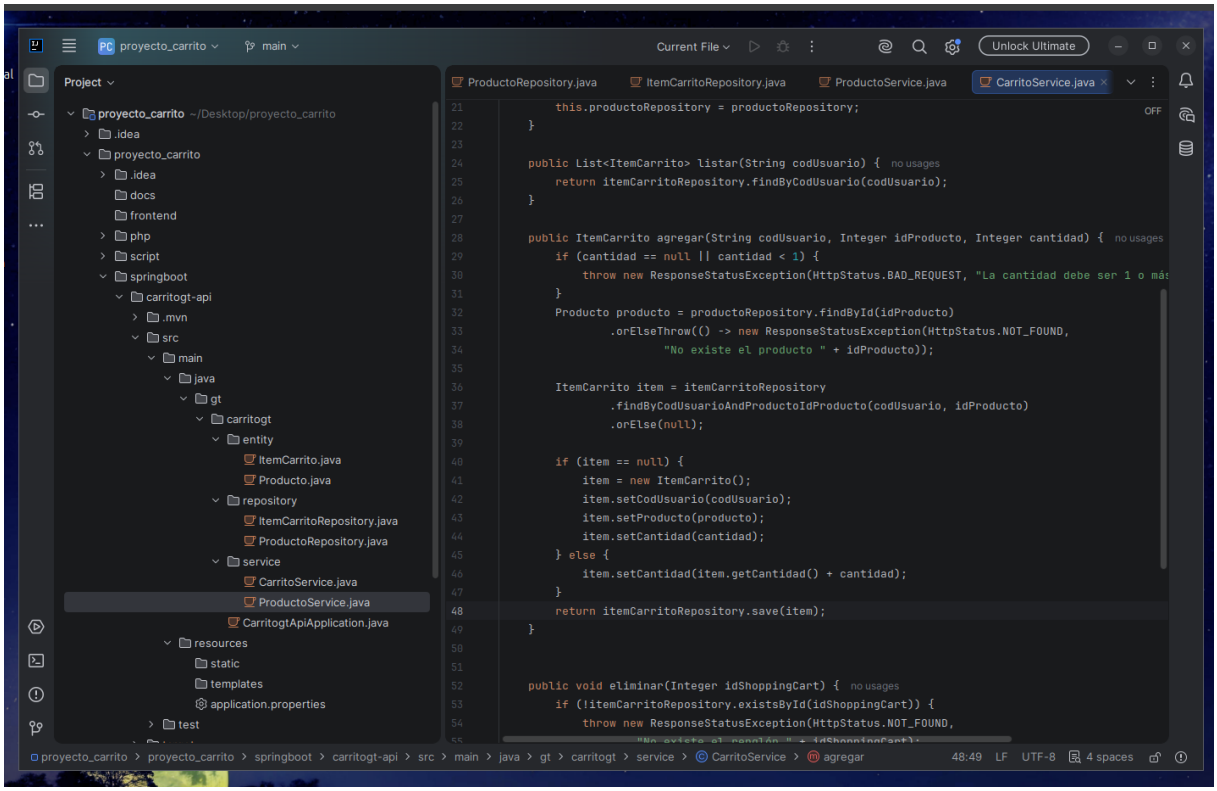


Figura 19. CarritoService.agregar: valida, busca el producto y suma o crea el renglón.

8. Diccionario de datos

Base de datos «cc» en MySQL 8.0, con las dos tablas del script que proporcionó el catedrático (script/cc.sql). Los tipos salen de DESCRIBE producto y DESCRIBE shopping_cart.

Tabla	Campo	Tipo	Clave	Descripción
producto	idProducto	int	PK, autoincremental	Identificador del producto
producto	nombre	varchar(255)		Nombre del producto
producto	codigo	varchar(255)	NOT NULL	Código del producto
producto	imagen	text		Ruta de la imagen (por ejemplo images/product/camera.jpg)
producto	precio	double(10,2)		Precio unitario en quetzales
shopping_cart	idShoppingCart	int	PK, autoincremental	Identificador del renglón del carrito
shopping_cart	idProducto	int	FK (MUL) → producto.idProduct	Producto agregado al carrito

			o	
shopping_cart	codUsuario	varchar(128)		Código que identifica al usuario (un GUID en PHP; un código guardado en el navegador en la página nueva)
shopping_cart	cantidad	int		Cantidad de unidades

Relación: un producto puede aparecer en muchos renglones del carrito (1 a muchos) y cada renglón pertenece a un solo producto, mediante la clave foránea idProducto. En el código, esa relación es @ManyToOne en ItemCarrito.

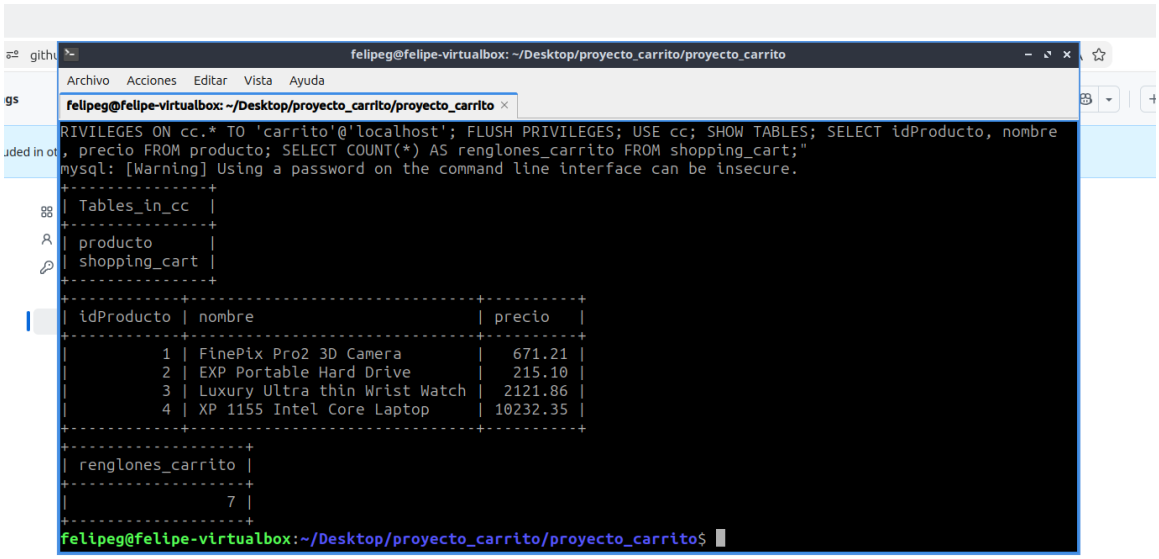


Figura 20. Tablas de la base cc y los 4 productos cargados.

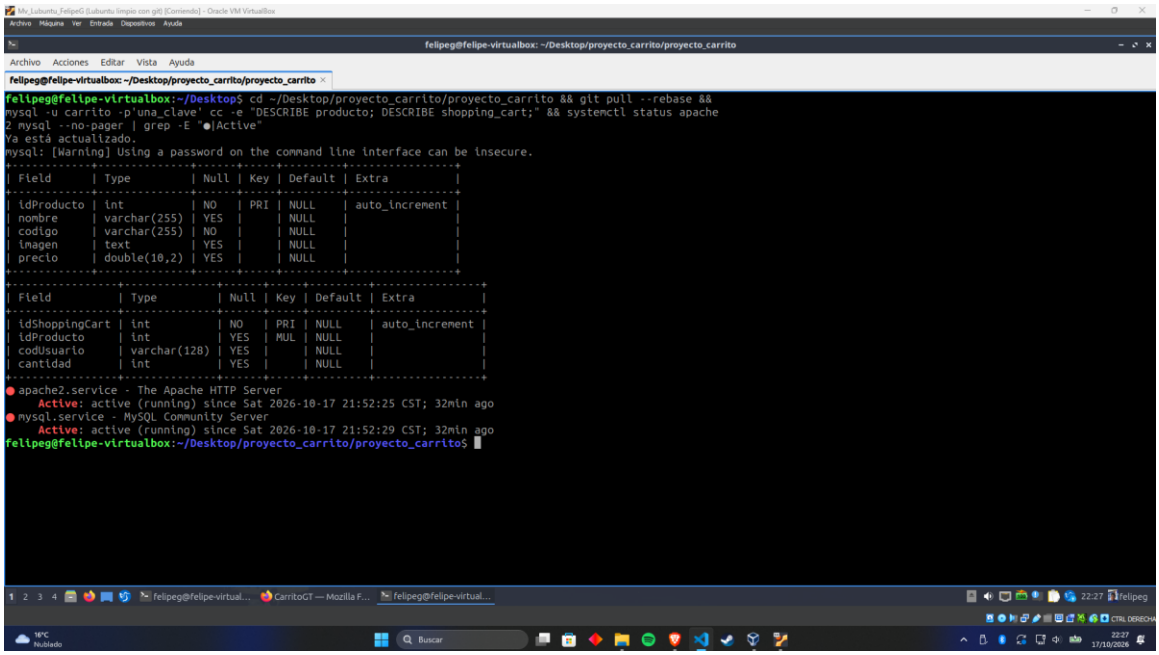


Figura 21. DESCRIBE de las dos tablas.

idProducto	Nombre	Código	Imagen	Precio (Q)
1	FinePix Pro2 3D Camera	3DcAM01	images/product/camera.jpg	671.21
2	EXP Portable Hard Drive	USB02	images/product/external-hard-drive.jpg	215.10
3	Luxury Ultra thin Wrist Watch	wristWear03	images/product/watch.jpg	2121.86
4	XP 1155 Intel Core Laptop	LPN45	images/product/laptop.jpg	10232.35

9. Contrato básico de la API

La API corre en http://localhost:8080 y responde JSON. Los errores devuelven un JSON con el estado y un mensaje comprensible (spring.web.error.include-message=always).

Método y dirección	Propósito	Parámetros	Respuesta
GET /api/productos	Lista los productos	ninguno	200 y lista JSON
GET /api/carrito/{codUsuario}	Lista el carrito del usuario	codUsuario (ruta)	200 y lista JSON (puede ser vacía)
POST /api/carrito/{codUsuario}	Agrega un producto; si ya estaba, suma la cantidad	codUsuario (ruta); cuerpo JSON {"idProducto": 2, "cantidad": 1}	201 con el renglón; 400 si la cantidad es menor que 1; 404 si el

			producto no existe
DELETE /api/carrito/item/{idShoppingCart}	Elimina un renglón del carrito	idShoppingCart (ruta)	204 sin cuerpo; 404 si no existe
DELETE /api/carrito/{codUsuario}	Vacía el carrito del usuario	codUsuario (ruta)	204 sin cuerpo

Ejemplo de respuesta de GET /api/productos (recortado):


```
[{"codigo":"3DcAM01","idProducto":1,"imagen":"images/product/camera.jpg","nombre":"Fine Pix Pro2 3D Camera","precio":671.21},
 {"codigo":"USB02","idProducto":2,"imagen":"images/product/external-hard-drive.jpg","nombre":"EXP Portable Hard Drive","precio":215.1}, ...]
```


Ejemplo de respuesta de POST /api/carrito/prueba1 con {"idProducto":2,"cantidad":1} (HTTP 201):


```
{"cantidad":1,"codUsuario":"prueba1","idShoppingCart":12,"producto":{"codigo":"USB02","idProducto":2,"imagen":"images/product/external-hard-drive.jpg","nombre":"EXP Portable Hard Drive","precio":215.1}}
```


Ejemplo de error (HTTP 404) al agregar un producto que no existe:


```
{"timestamp":"2026-10-18T03:28:25.041Z","status":404,"error":"Not Found","message":"No existe el producto 99","path":"/api/carrito/prueba1"}
```


10. Evidencia de pruebas

Probé cada operación de la API con curl antes de conectar la página, como pide el enunciado. Usé el usuario de prueba «prueba1».

Prueba	Resultado esperado	Resultado obtenido
GET /api/productos	200 con 4 productos	200, 4 productos en JSON
POST agregar producto 2, cantidad 1	201	201, renglón creado
POST agregar de nuevo producto 2, cantidad 2	201 y suma	201, cantidad 3 (se sumó, no se duplicó)
GET /api/carrito/prueba1	200 con un solo renglón de cantidad 3	200, un renglón con cantidad 3
POST con cantidad 0	400	400 «La cantidad debe ser 1 o más»
POST con producto 99	404	404 «No existe el producto 99»
DELETE /api/carrito/prueba1	204	204 sin cuerpo

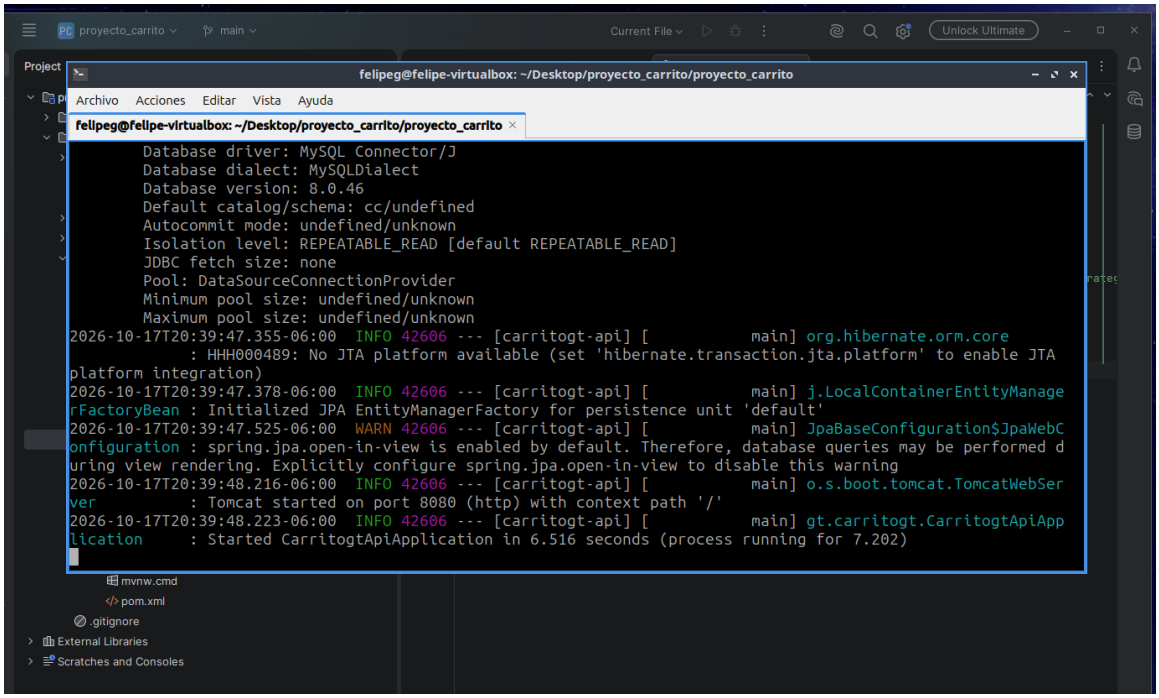


Figura 22. Spring Boot arranca y se conecta a MySQL 8.0.46 (base cc); Tomcat en el puerto 8080.

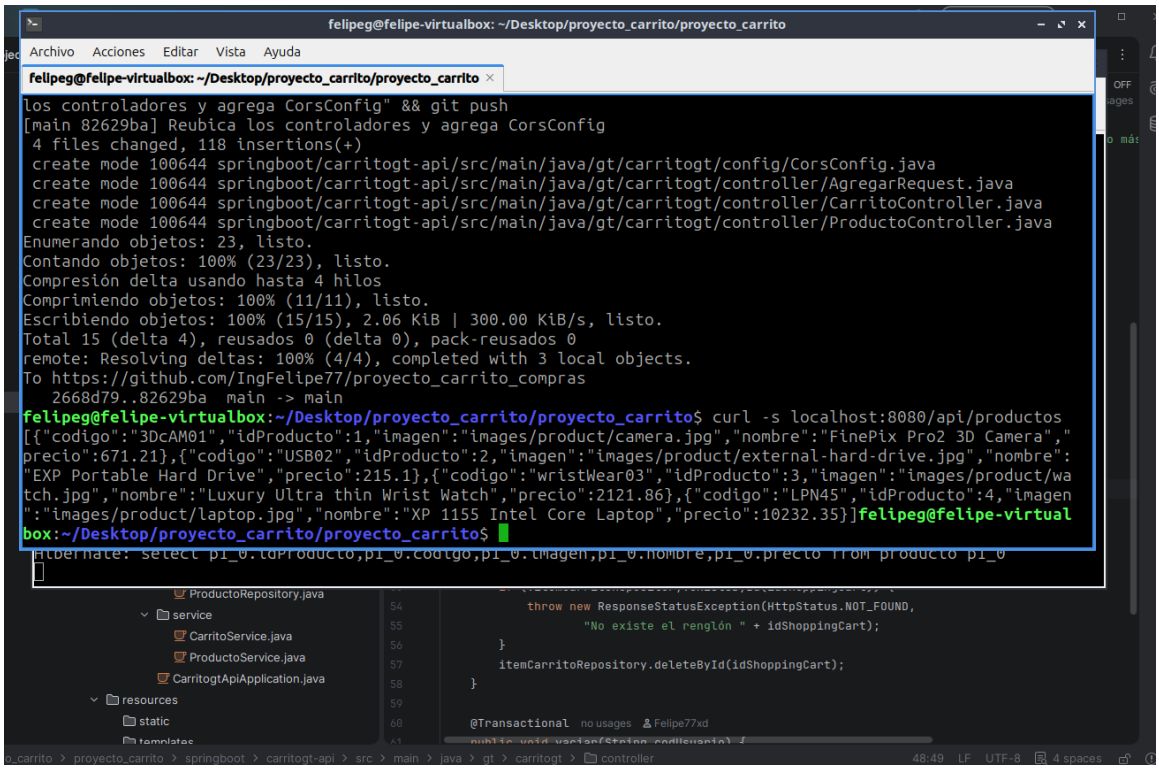


Figura 23. GET /api/productos devuelve los 4 productos; en la otra terminal se ve el SQL que Spring generó.

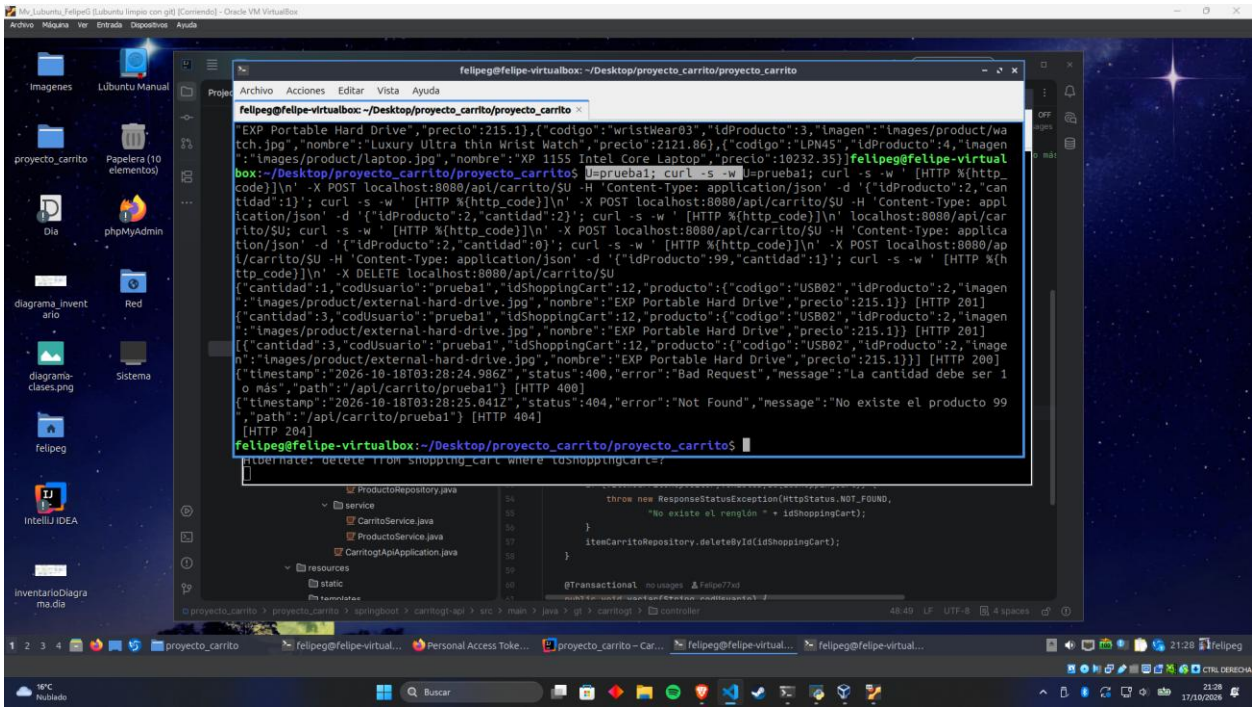


Figura 24. Pruebas con curl: 201, suma a cantidad 3, 200, 400, 404 y 204.

Prueba de extremo a extremo: con la API corriendo, abrí http://carritogt.localhost. La página lista los productos desde la API, agrega (y suma), muestra el total y el contador de unidades, elimina un renglón y vacía el carrito.

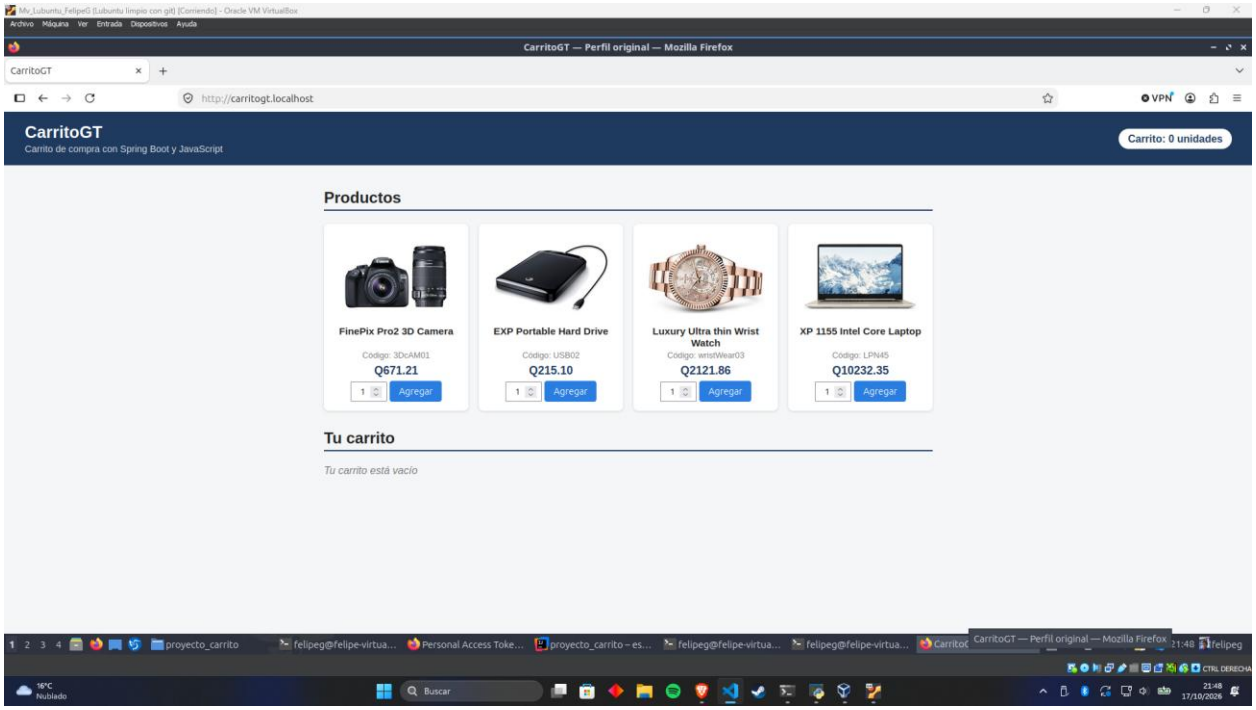


Figura 25. Página en carritogt.localhost: los 4 productos llegan desde la API con fetch.

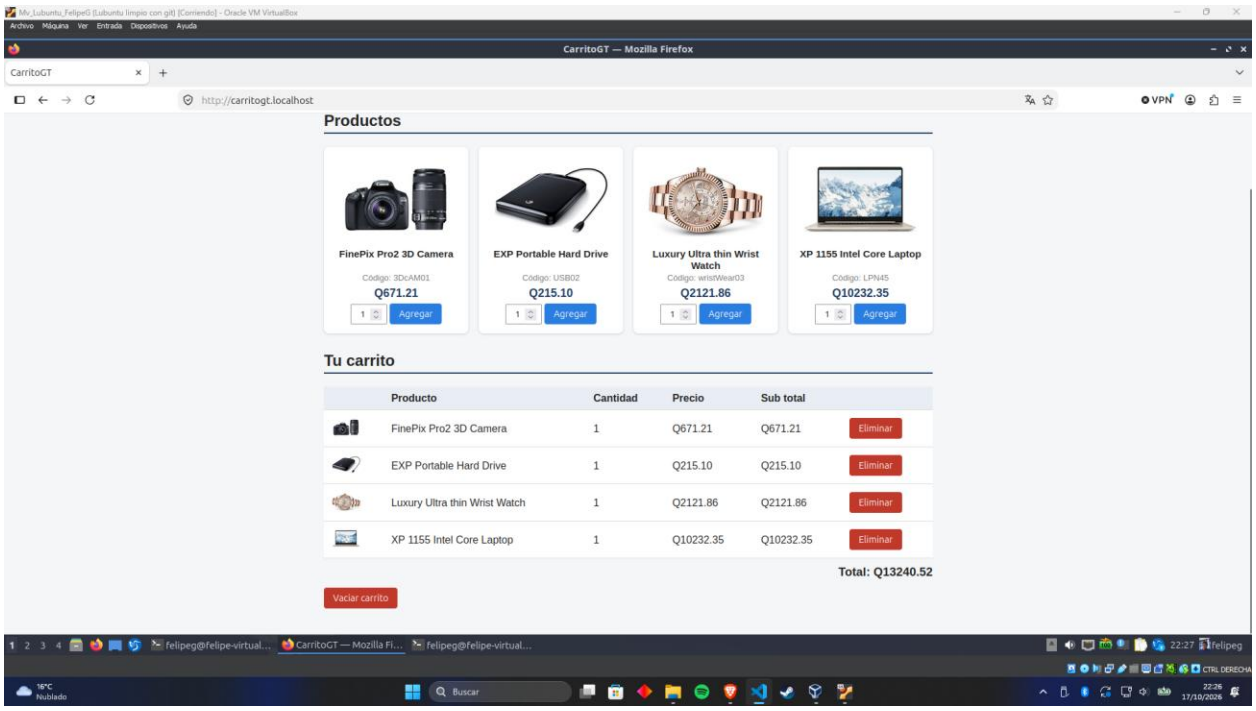


Figura 26. Página en carritogt.localhost con el carrito lleno, consumiendo la API con fetch.

11. Manual de usuario

Para iniciar el entorno:

1. Abrir VirtualBox e iniciar la máquina virtual «Mv_Lubuntu_FelipeG».
2. En una terminal, comprobar los servicios: systemctl is-active apache2 mysql (deben decir active; si no, sudo systemctl start apache2 mysql).
3. Iniciar la API: cd ~/Desktop/proyecto_carrito/proyecto_carrito/springboot/carritogt-api && mvn spring-boot:run. Esperar a ver «Started CarritogtApiApplication».
4. Abrir Firefox en http://carritogt.localhost (versión nueva, con la API) o en http://carrito-compra.localhost (versión PHP).

Para usar el carrito (versión nueva):

- Ver productos: aparecen al abrir la página, con imagen, código y precio.
- Agregar al carrito: escribir la cantidad y pulsar «Agregar». Si el producto ya estaba, se suma la cantidad.
- Ver el carrito: la sección «Tu carrito» muestra los renglones, el subtotal, el total y el contador de unidades del encabezado.
- Eliminar: pulsar «Eliminar» en el renglón.
- Vaciar: pulsar «Vaciar carrito».

Para detener la API: Ctrl+C en la terminal donde corre.

12. Bitácora de desarrollo

Fecha	Avance realizado	Problema o decisión	Solución / aprendizaje	Commit
16/10/2026	Revisé la VM Lubuntu y lo ya instalado (Apache, MySQL 8.0, Java 21, Maven, Git)	La VM traía PHP 8.3 y el repo base pide PHP 7 (RNF-04); el manejador de errores del repo convierte avisos en errores fatales	Instalé PHP 7.4 junto al 8.3 con el PPA ondrej/php, a2dismod php8.3 y a2enmod php7.4; php -v = 7.4.33	—
16/10/2026	Intento de instalar PHP 7.4	«El paquete php7.4 no está disponible»: faltaba el repositorio extra	Agregué ppa:ondrej/php y actualicé la lista	—
16/10/2026	Instalación de PHP 7.4	php-common rompía a php-mcrypt	Comprobé que el proyecto no usa mcrypt y lo desinstalé	—
17/10/2026	Diagramas de casos de uso y de clases (plan) en Dia	Dibujé primero el diagrama, como pide el ingeniero, antes de programar	Clases por capas: entity, repository, service y controller, con toString y multiplicidad	—
17/10/2026	Ubiqué mi repositorio y traje el código del ingeniero	La carpeta php-mvc de la VM era de otro autor (daveh), no el carrito del ingeniero	Cloné mvelasquezo/carrito-compra-basico y lo copié a php/ de mi repo; script de la base en script/cc.sql	d1668c5
17/10/2026	Primer push de mi repo desde la VM	GitHub rechazó mi contraseña (autenticación por contraseña no soportada)	Creé un token personal (classic, scope repo) y lo guardé en la caché de Git por 8 horas	d1668c5
17/10/2026	MySQL: preparar la base cc y el usuario carrito	No recordaba la contraseña de root («Access denied»); al reiniciar MySQL sin permisos	Restablecí la contraseña de root, maté el proceso viejo y arranqué el servicio normal; la base cc ya traía las 2 tablas y los 4 productos; creé el usuario carrito con permisos solo sobre cc	—

		quedó un proceso viejo bloqueando ibdata1 (error 11)		
17/10/2026	PHP funcionando en su Virtual Host	Config.php usaba root/USAC123 y HOST carrito-compra.com; los Virtual Hosts viejos apuntaban a otra carpeta	Cambié el usuario a carrito, HOST a carrito-compra.localhost, reescribí el Virtual Host hacia php/public, chmod o+x en mi carpeta y escribible php/logs; probé ver, agregar y eliminar	—
17/10/2026	Creé el proyecto Spring Boot 4.1.1 (Java 21, Maven, Web, JPA, MySQL)	Initializr puede generar la versión 4.1.1.RELEASE, que no existe en Maven; la correcta es 4.1.1	Conecté con application.properties: ddl-auto=none y naming StandardImpl; la app arrancó y se conectó a cc	6cb47a6
17/10/2026	Capas Entity, Repository y Service	—	Producto e ItemCarrito con @ManyToOne; repositorios con consultas derivadas; CarritoService.agregar suma si ya existe (400 y 404 con ResponseStatusException)	f07ee32, 2668d79
17/10/2026	Controladores REST y CORS	Las carpetas controller y config quedaron anidadas por error en IntelliJ	Las reubiqué con la terminal; probé con curl 201, suma, 200, 400, 404 y 204	82629ba
17/10/2026	Página con fetch en carritogt.localhost	El app.js no se había creado y la página quedaba vacía (404 del archivo)	Creé frontend/js/app.js; la página lista, agrega, suma, elimina y vacía consumiendo la API	4f831a8
17/10/2026	Documentación: README, bitácora, diagramas, capturas y este	—	Subí la documentación al repositorio	—

	documento			

Fuentes consultadas y uso de IA: documentación de Ubuntu, MySQL y Spring; el enunciado del proyecto y las convenciones del curso. Usé asistencia de inteligencia artificial generativa (Claude) como tutor para entender conceptos (capas, anotaciones de Spring, CORS, fetch) y para que me guiara en los pasos de la máquina virtual. Escribí y ejecuté el código en mi propia máquina virtual, y estoy preparado para explicarlo y modificarlo en la defensa.

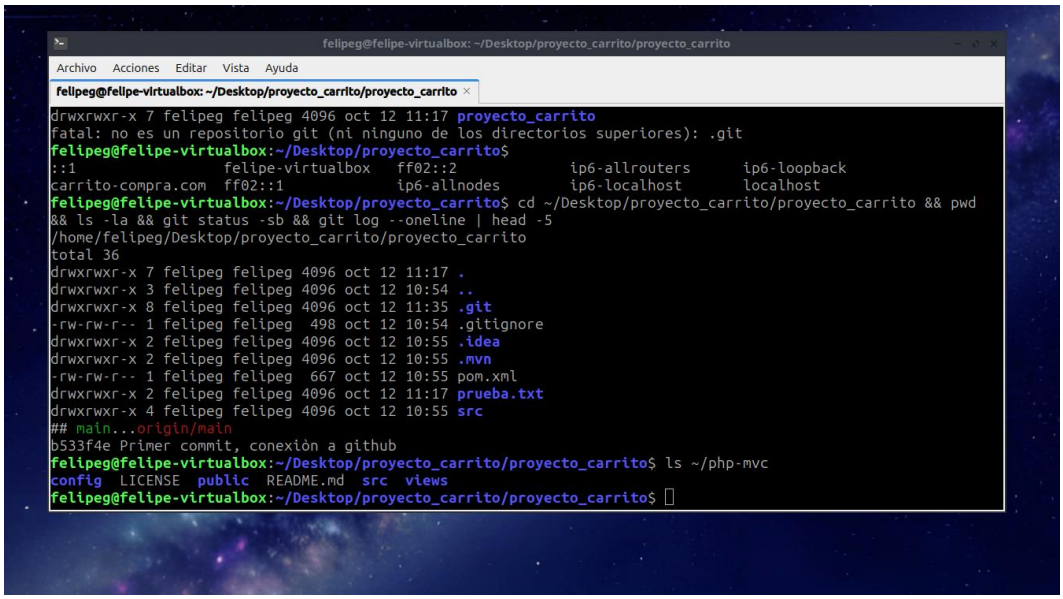


Figura 27. Repositorio de Git enlazado a GitHub.

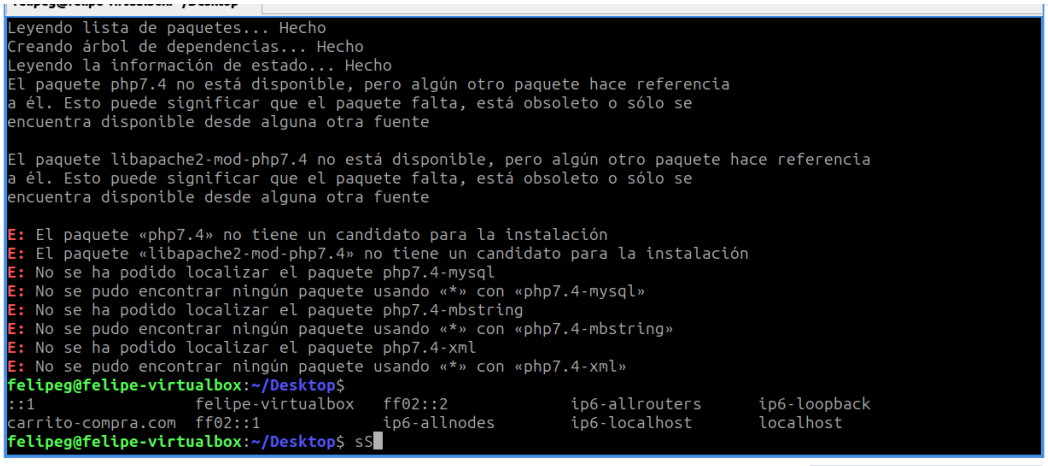


Figura 28. Error: php7.4 no disponible.

```
felipeg@felipe-virtualbox:~/Desktop$ sudo apt install -y php7.4 libapache2-mod-php7.4 php7.4-mysql php7.4-mb
string php7.4-xml && (sudo a2dismod php8.3; sudo a2enmod php7.4; sudo update-alternatives --set php /usr/bin
/php7.4; sudo systemctl restart apache2; echo "=== RESULTADO ==="; php -v | head -1; apache2ctl -M 2>/dev/nu
ll | grep -i php)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
No se pudieron instalar algunos paquetes. Esto puede significar que
usted pidió una situación imposible o, si está usando la distribución
inestable, que algunos paquetes necesarios aún no se han creado o se
han sacado de «Incoming».
La siguiente información puede ayudar a resolver la situación:

Los siguientes paquetes tienen dependencias incumplidas:
 php-common : Rompe: php-mcrypt (< 3:1.0.7-5~)
E: Error, pkgProblemResolver::Resolve generó cortes, esto puede deberse a paquetes retenidos.
felipeg@felipe-virtualbox:~/Desktop$
::1               felipe-virtualbox   ff02::2            ip6-allrouters      ip6-loopback
carrito-compra.com  ff02::1           ip6-allnodes       ip6-localhost       localhost
felipeg@felipe-virtualbox:~/Desktop$
```

Figura 29. Error: php-common rompe php-mcrypt.

```
Los siguientes paquetes tienen dependencias incumplidas:
 php-common : Rompe: php-mcrypt (< 3:1.0.7-5~)
E: Error, pkgProblemResolver::Resolve generó cortes, esto puede deberse a paquetes retenidos.
felipeg@felipe-virtualbox:~/Desktop$
::1               felipe-virtualbox   ff02::2            ip6-allrouters      ip6-loopback
carrito-compra.com  ff02::1           ip6-allnodes       ip6-localhost       localhost
felipeg@felipe-virtualbox:~/Desktop$ apt-get -s remove php-mcrypt | grep -E "^(Remv|Inst)"
Remv php-mcrypt [3:1.0.5-4ubuntu1]
felipeg@felipe-virtualbox:~/Desktop$ apt-get -s remove php-mcrypt | grep -E "^(Remv|Inst)"
Remv php-mcrypt [3:1.0.5-4ubuntu1]
felipeg@felipe-virtualbox:~/Desktop$ sudo apt remove -y php-mcrypt
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los siguientes paquetes se ELIMINARÁN:
  php-mcrypt
0 actualizados, 0 nuevos se instalarán, 1 para eliminar y 116 no actualizados.
Se liberarán 11.3 kB después de esta operación.
(Leyendo la base de datos ... 372936 ficheros o directorios instalados actualmente.)
Desinstalando php-mcrypt (3:1.0.5-4ubuntu1) ...
felipeg@felipe-virtualbox:~/Desktop$
```

Figura 30. php-mcrypt eliminado.